==========

Projet : Recherche de chemin pour un robot

==========

OBJECTIF :

Programmer un robot qui se déplace dans une ville modélisée par une grille,

et trouver le meilleur chemin entre un point de départ (S) et une destination (G).

Comparer plusieurs algorithmes heuristiques : Glouton, A*, Algorithme Génétique.

GRILLE :

Exemple :

S 0 0 0 X 0

0 X 0 0 X 0

0 X 0 0 0 0

0 0 0 X 0 G

- S : Départ
- G : Destination
- 0 : Case libre
- X : Obstacle

Chaque case est représentée par ses coordonnées (ligne, colonne) dans la grille.

FONCTIONS UTILES :

---

1. get_neighbors(grid, x, y)
   - Renvoie les cases accessibles depuis (x, y)
   - Ignore les obstacles et les positions hors grille

2. manhattan(a, b)

- Distance de Manhattan entre deux positions
- Utilisée comme heuristique

ALGORITHMES :

1.Algorithme Glouton :

- Principe : choisir à chaque étape le voisin le plus proche de l'objectif selon l'heuristique.
- Avantage : très rapide.
- Limite : ne garantit pas le chemin optimal et peut se bloquer.
- Exemple :

```
path = greedy_search(grid, start, goal)
```

2.Algorithme A* :

- Principe : f(n) = g(n) + h(n)
  - g(n) : coût depuis le départ
  - h(n) : estimation du coût pour atteindre l'objectif (heuristique)
- Explore intelligemment les chemins les plus prometteurs
- Garantit un chemin optimal si l'heuristique ne surestime pas
- Exemple :

```
path = a_star(grid, start, goal)
```

3.Algorithme Génétique :

- Représentation : un chemin est une séquence de mouvements [H, B, G, D]
- Étapes :
  1. Générer une population de chemins aléatoires
  2. Calculer la fitness (proximité de l'objectif + pénalité obstacles)
  3. Sélection, croisement, mutation
  4. Répéter jusqu'à obtenir un chemin satisfaisant

- Avantages : flexible, peut explorer des labyrinthes complexes

- Limites : plus lent, pas garanti optimal

- Exemple :

```
path = genetic_algorithm()
```

AFFICHAGE :

- Affichage console avec '*' pour le chemin.

- Vérifier si le chemin existe avant d'afficher (sinon "Aucun chemin trouvé").

BIBLIOTHÈQUES :

- Standard Python : heapq, random, time

- Optionnelles pour visualisation : matplotlib (grille colorée), pygame (animation)

ANALYSE ET COMPARAISON :

Algorithme	Longueur chemin	Temps	Commentaire
Glouton	Variable	Très rapide	Rapide mais peut échouer
A*	Optimal	Moyen	Fiable et efficace
Génétique	Approximatif	Lent	Flexible mais aléatoire

CONSEILS :

- Toujours tester avec plusieurs grilles

- Mesurer le temps d'exécution pour comparaison

- Observer comment chaque algorithme réagit aux obstacles

- Utiliser print ou matplotlib pour visualiser les chemins

==========